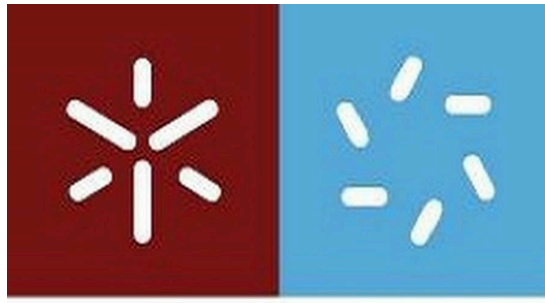




Universidade do Minho
Escola de Ciências

Computação Gráfica

2024/2025

Trabalho Prático

Fase 2 - Transformações Geométricas

Grupo 12

Afonso Martins n°A102940

Luis Silva n° A105530

Pedro Gomes n° A102929

Ricardo Vilaça n°A102879

Contents

1. Introdução	3
2. Implementação de transformações e hierarquia de grupos	3
2.1. Parsing dos ficheiros XML	3
2.1.1. Transformation	3
2.1.2. Group	3
2.2. Desenho	4
3. Definição do Sistema Solar:	4
3.1. Formato do Ficheiro XML:	5
3.2. Implementação do Torus	8
3.3. Sistemas Solares Finais:	10
4. Orbital (Extra)	12
5. Conclusão	13

1. Introdução

Nesta fase seguinte foi pedido para criar cenas hierárquicas utilizando transformações geométricas. A engine precisa de ser atualizada para suportar este tipo de estrutura. Cada cena é representada como uma árvore, onde em cada *node* contém um grupo de transformações geométricas (translação, rotação e escala) onde pode conter um ou mais modelos. Para além disto, cada *node* filho pode ter *nodes* filhos, que permite a criação de hierarquias que possibilitam representações mais complexas e organizadas dentro da cena.

2. Implementação de transformações e hierarquia de grupos

2.1. Parsing dos ficheiros XML

Com a implementação de grupos e transformações na gramática do xml foi necessário fazer alterações a engine para que fosse possível realizar essas transformações de forma hierárquica. Para tal efeito procedeu-se à criação de 2 novas *structs* dentro da engine, *Transformation* e *Group*.

2.1.1. Transformation

Esta *struct* define um dos três tipos de transformações, sendo a mais simples e composta por 4 floats: **x, y, z e angle** e por um *enum*, *Type* que agrupa as constantes *TRANSLATE*, *ROTATE* e *SCALE*.

```
struct Transformation {  
    Type type;  
    float x, y, z, angle = 0;  
};
```

Figure 1: *Struct Transformation*

```
enum Type {  
    TRANSLATE,  
    ROTATE,  
    SCALE  
};
```

Figure 2: *Enum Type*

2.1.2. Group

A *struct Group* guarda os conteúdos do ficheiro xml, sendo composto por 3 listas: uma lista com todas as transformações (tipo *Transformation*) a executar para o grupo atual, uma lista com todos os ficheiros a serem desenhados pelo grupo atual (tipo *string*) e uma outra lista onde são guardados os sub-grupos do grupo atual (tipo *Group*).

```
struct Group {  
    list<string> files;  
    list<Transformation> transformations;  
    list<Group> children;  
};
```

Figure 3: *Struct Group*

2.2. Desenho

Para desenhar as figuras com uma hierarquia, nós optamos por desenhar todas as figuras de um grupo e só depois passamos para o próximo. Isto implica que todos os sub-grupos desse grupo sejam de mesmo modo completamente desenhados. Para regular as transformações e a herança das transformações do grupo pai, recorremos a função *glPushMatrix()*, usada antes do desenho de qualquer grupo exceto o grupo principal, e a função *glPopMatrix()*, usada após o final do grupo atual para evitar transformações indesejadas em grupos *irmãos*.

3. Definição do Sistema Solar:

O sistema solar foi pedido a forma de demonstrar as funcionalidades que foram desenvolvidas nesta 2ª fase. O início desta secção foi iniciada com uma inspiração na aplicação *paint*, como podemos ver na figura abaixo

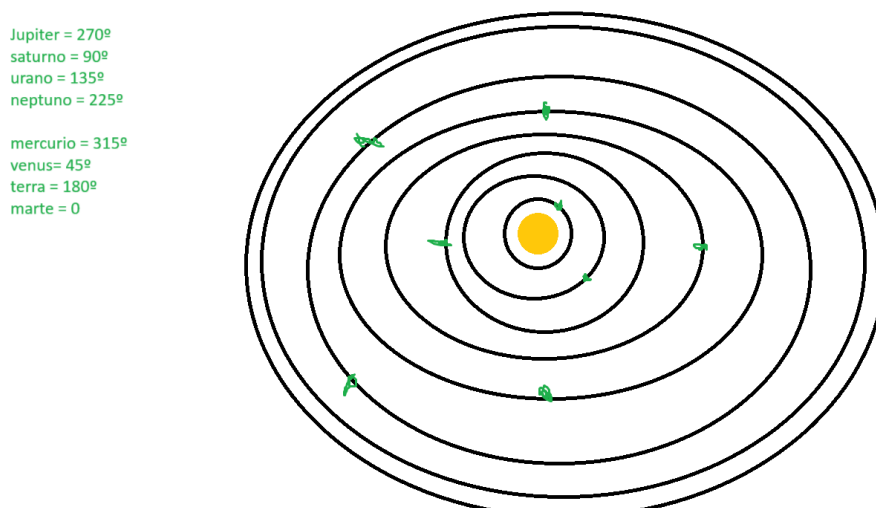


Figure 4: Base Para o Sistema Solar

Esta figura serviu como base de inspiração para a criação do sistema solar. Os valores à esquerda são os graus de cada planeta para executar a seguinte fórmula para a definição de um ponto com coordenadas(x,y,z), onde y = 0:

$$x = \cos(\theta) * r$$

$$z = \sin(\theta) * r$$

Sendo r o raio da esfera(planeta) e θ um ângulo definido para cada planeta, neste caso vamos usar:

- Mercúrio -> $\theta = 315^\circ$
- Vénus -> $\theta = 45^\circ$
- Terra -> $\theta = 180^\circ$
- Marte -> $\theta = 0^\circ$
- Jupiter -> $\theta = 270^\circ$
- Saturno -> $\theta = 90^\circ$
- Urano -> $\theta = 135^\circ$
- Néptuno -> $r = e$ $\theta = 225^\circ$

Com estes cálculos podemos assim definir a posição de cada planeta:

- Mercúrio(30,0,-30):

$$X = \cos(315) * 43 = 30 \mid Z = \sin(315) * 43 = -30$$

- Vénus(46,0,0):

$$X = \cos(45) * 65 = 46 \mid Z = \sin(45) * 65 = 46$$

- Terra(-85,0,0):

$$X = \cos(180) * 85 = -85 \mid Z = \sin(180) * 85 = 0$$

- Marte(100,0,0):

$$X = \cos(0) * 100 = 100 \mid Z = \sin(0) * 100 = 0$$

- Júpiter(0,0,-125):

$$X = \cos(270) * 125 = 0 \mid Z = \sin(270) * 125 = -125$$

- Saturno(0,0,170):

$$X = \cos(90) * 170 = 0 \mid Z = \sin(90) * 170 = 170$$

- Urano(-145,0,145):

$$X = \cos(135) * 205 = -145 \mid Z = \sin(135) * 205 = 145$$

- Neptuno(166,0,166):

$$X = \cos(225) * 235 = 166 \mid Z = \sin(225) * 235 = 166$$

3.1. Formato do Ficheiro XML:

O ficheiro XML desenvolvido o Sistema Solar é composto por vários grupos que representam o Sol ou planetas. Estes podem ser compostos por subgrupos para representar astros, satélites... no nosso sistema decidimos fazer a Lua. Para a geração dos ficheiros .3d utilizamos o comando:

.generate sphere raio stacks slices nomeDoFicheiro.3d

Este comando vai criar um ficheiro .3d na pasta *models* que tem todos os modelos utilizados no projeto.

O Sol foi desenhado/gerado no centro do referencial(coordenadas $x = 0$, $y = 0$ e $z = 0$), assim como podemos ver na seguinte figura:

```

<group>
  <!-- Sol -->
  <group>
    <transform>
      <scale x="20" y="20" z="20" />
    </transform>
    <models>
      <model file="sphere.3d" />
    </models>
  </group>

```

Figure 5: Ficheiro XML da geração do Sol no centro do referencial

Para os planetas, seguiram uma estrutura similar:

```

<!-- Júpiter -->
<group>
  <transform>
    <rotate angle="0" x="0" y="1" z="0" />
    <translate x="80" y="0" z="-80" />
    <scale x="11.21" y="11.21" z="11.21" />
  </transform>
  <models>
    <model file="sphere.3d" />
  </models>
</group>

```

Figure 6: Ficheiro XML da Geração de Júpiter

Relativamente à geração dos planetas com satélites como a Terra que possui a Lua como satélite natural, é necessária uma translação para ser posicionada relativamente ao Sol, e de seguida é utilizado dois subgrupos, um para desenhar a Terra e outro a Lua. Assim como a Terra sofreu uma translação relativamente ao Sol a Lua também vai sofrer uma translação relativamente à Terra. Como é demonstrado na figura abaixo:

```

<!-- Terra -->
<group>
  <transform>
    <rotate angle="0" x="0" y="1" z="0" />
    <translate x="-60" y="0" z="0" />
    <scale x="1.4" y="1.4" z="1.4"/>
  </transform>
  <models>
    <model file="sphere.3d" />
  </models>

  <!-- Lua -->
  <group>
    <transform>
      <rotate angle="0" x="0" y="1" z="0" />
      <translate x="3" y="0" z="2" />
      <scale x="0.27" y="0.27" z="0.27" />
    </transform>
    <models>
      <model file="sphere.3d" />
    </models>
  </group>
</group>

```

Figure 7: Ficheiro XML da Geração da Terra e a Lua

Relativamente à geração de Saturno, tal como na geração da Terra e da Lua, foi preciso realizar uma translação para posicionar o planeta corretamente em relação ao Sol, e de seguida é utilizado dois subgrupos, um para desenhar Saturno e outro especificamente para gerar o seu anel através da primitiva torus. Como é demonstrado na figura abaixo:

```

<!-- Saturno -->
<group>
  <transform>
    <rotate angle="0" x="0" y="1" z="0" />
    <translate x="0" y="0" z="100" />
    <scale x="9.45" y="9.45" z="9.45" />
  </transform>
  <models>
    <model file="sphere.3d" />
  </models>

  <!-- Anel de Saturno -->
  <group>
    <transform>
      <rotate angle="30.0f" x="1" y="0" z="1" />
      <translate x="0" y="0" z="0" />
      <scale x="0.5" y="0.5" z="0.5" />
    </transform>
    <models>
      <model file="torus.3d" /> <!-- generator torus 3 16 2 20 torus.3d -->
    </models>
  </group>
</group>

```

Figure 8: Ficheiro XML da Geração de Saturno e o seu Anel

3.2. Implementação do Torus

Para complementar o modelo do planeta Saturno, adicionamos um anel utilizando a primitiva torus no gerador do nosso projeto. Esta primitiva é especialmente adequada pois permite representar formas anelares e pode ser ajustada facilmente através dos seus parâmetros.

Para gerar os pontos que compõem o torus, definimos o raio interno, o raio externo, e os números pretendidos de slices e stacks para detalhar a geometria. Especificamente para o anel de Saturno, utilizamos 2 stacks, permitindo obter a forma “achatada” característica do planeta. Com estes valores, calculamos as dimensões necessárias do torus da seguinte forma:

Raio interno (innerRadius): distância da parte interior até ao centro do anel.

Raio externo (outerRadius): distância do exterior até ao centro do anel.

Slices: divisões ao longo da circunferência principal.

Stacks: divisões ao longo da circunferência secundária, perpendicular à primeira.

Com estas informações, calculamos dois passos angulares:

Incremento angular entre slices: $\text{sliceStep} = 2\pi / \text{slices}$

Incremento angular entre stacks: $\text{stackStep} = 2\pi / \text{stacks}$

Deste modo, percorremos cada stack incrementando o ângulo β , gerando triângulos strip a strip. Dentro de cada stack, percorremos as slices incrementando o ângulo α , obtendo as coordenadas dos pontos para cada triângulo como segue:

$$x = (\text{outerRadius} + \text{innerRadius} \times \cos(\alpha)) \times \cos(\beta)$$

$$y = \text{innerRadius} \times \sin(\alpha)$$

$$z = (\text{outerRadius} + \text{innerRadius} \times \cos(\alpha)) \times \sin(\beta)$$

Segue abaixo uma figura ilustrativa do torus com 2 stacks, representando o anel achatado do planeta Saturno:

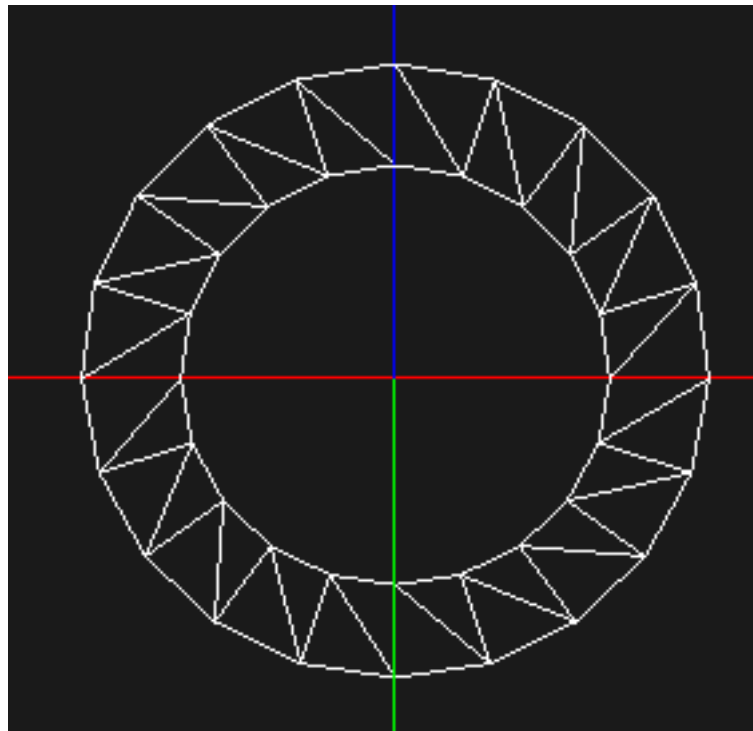


Figure 9: Torus

Após a realização da primitiva do torus com 2 stacks para a criação do anel de Saturno, adicionamos-na na geração de Saturno. Como podemos ver na figura abaixo:

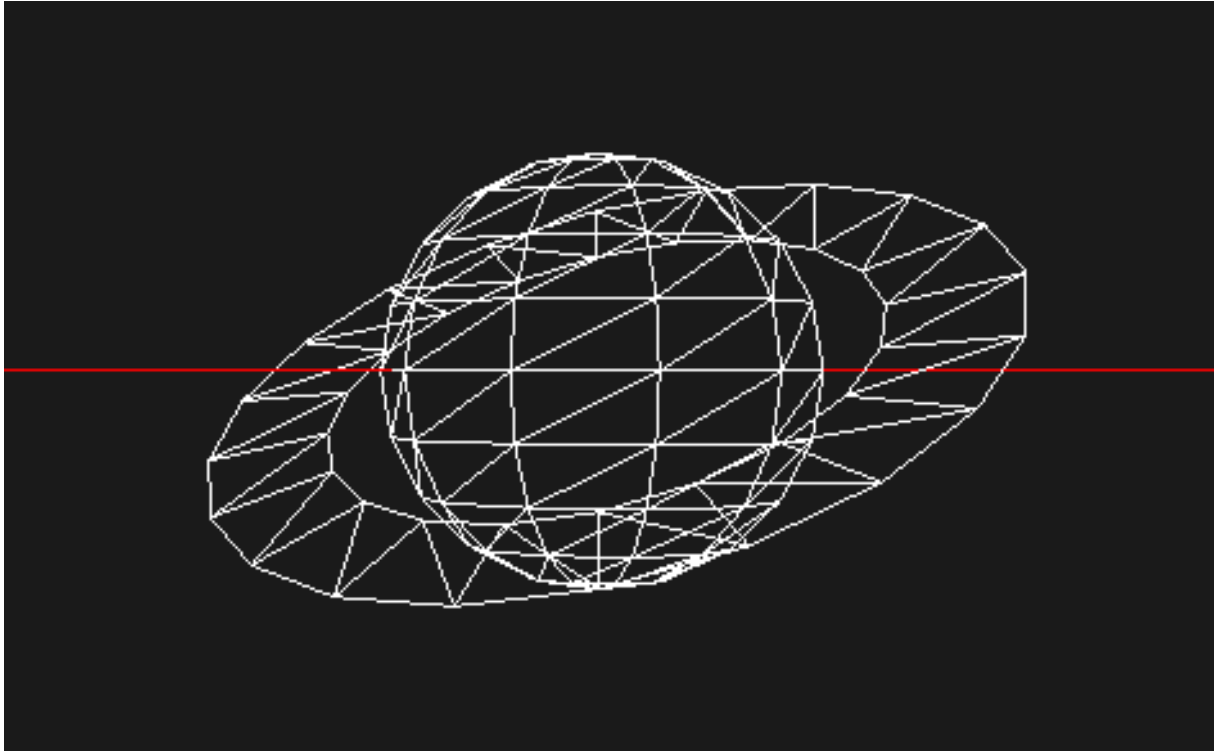


Figure 10: Saturno

3.3. Sistemas Solares Finais:

Através da nossa engine com o comando:

```
./engine tests sistemaSolar.xml
```

Nós somos capazes de produzir o seguinte Sistema solar:

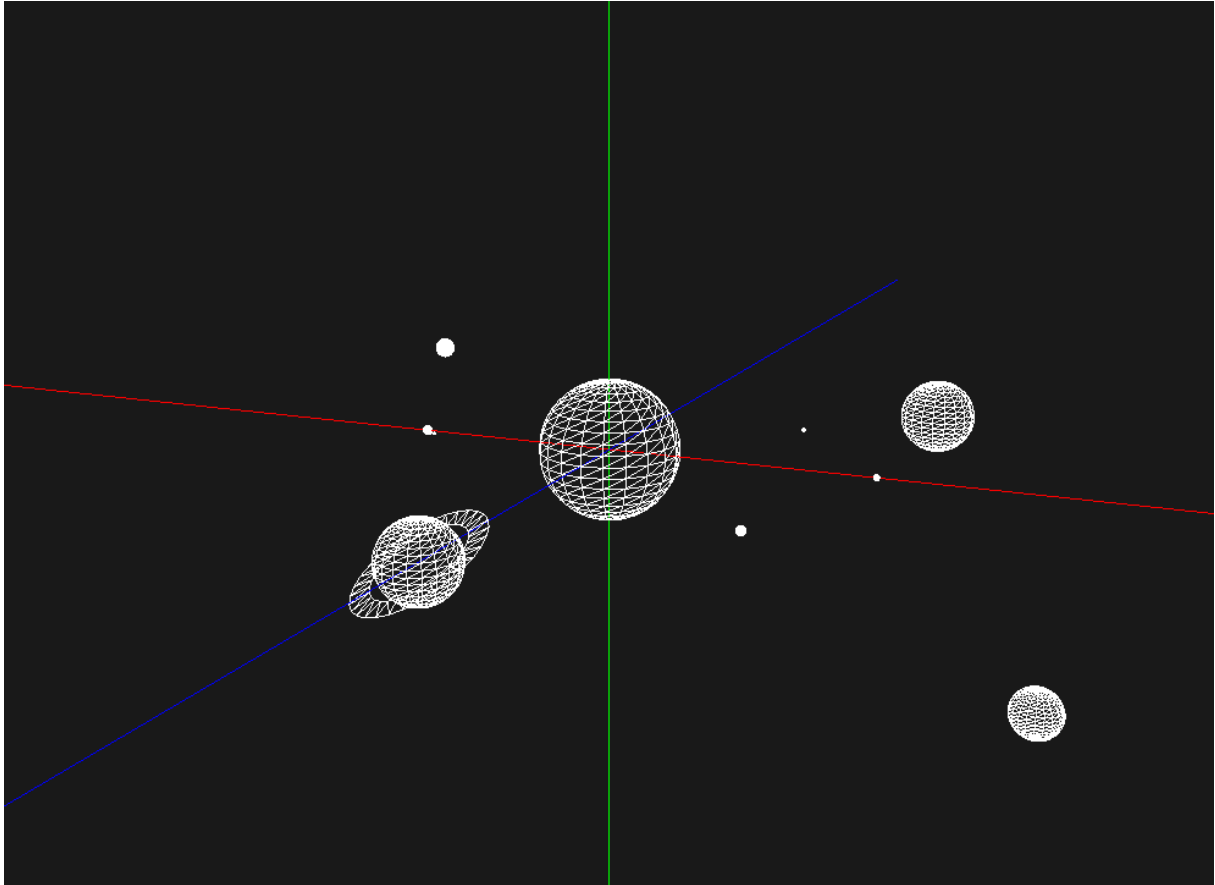


Figure 11: Sistema Solar

O grupo decidiu realizar outro Sistema Solar mas onde seja possível verificar distâncias relativas entre os planetas, para isto foi também elaborado outro ficheiro XML. Em que os planetas e o Sol ficaram todos alinhados no eixo do x

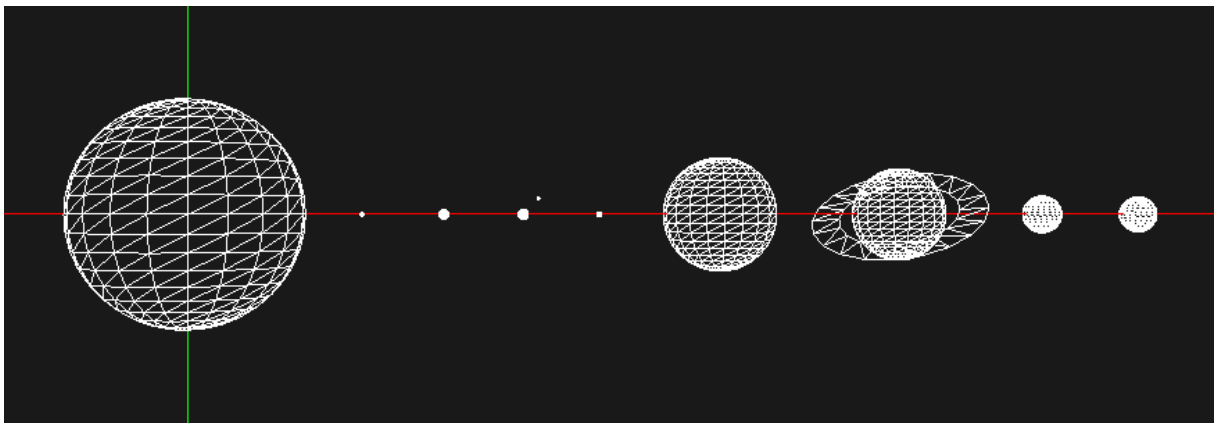


Figure 12: Sistema Solar Alinhado

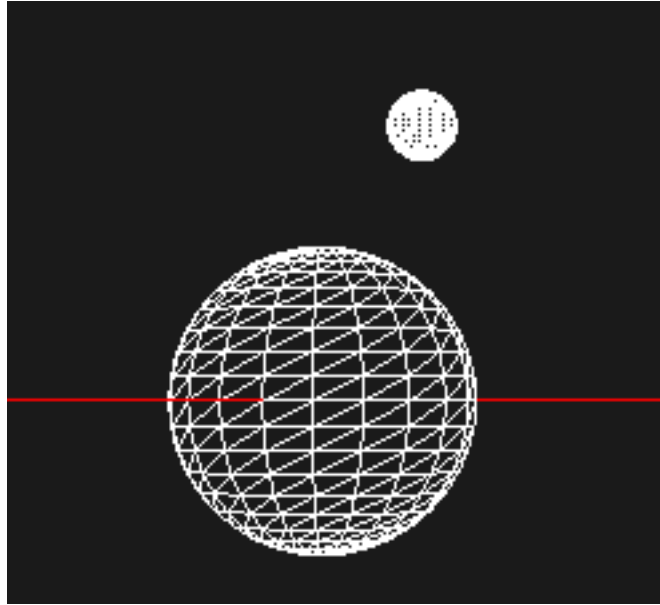


Figure 13: Imagem mais detalhada que mostra a Lua e a Terra

4. Orbital (Extra)

Para uma melhor visualização do sistema solar, decidimos adicionar uma câmera orbital focada no centro do sistema solar, ou seja, o sol. Esta câmera pode ser controlada através dos seguintes inputs do teclado:

- ↑ = Rotação da câmera para cima;
- ↓ = Rotação da câmera para baixo;
- ← = Rotação da câmera para a esquerda;
- → = Rotação da câmera para a direita;
- '+' = Dar Zoom In;
- '-' = Dar Zoom Out;

Para realizar estas rotações, foi necessário encontrar primeiro informações relevantes para a câmera, isto é, os ângulos alfa e beta da câmera, juntamente com o raio da sua orbital:

Coordenadas da posição inicial da câmera = (x,y,z)

$$raio = \sqrt{(x)^2 + (y)^2 + (z)^2}$$

$$alfa = \arcsin\left(\frac{x}{\sqrt{(x)^2 + (z)^2}}\right)$$

$$beta = \arcsin\left(\frac{y}{\sqrt{(x)^2 + (y)^2 + (z)^2}}\right)$$

5. Conclusão

Nesta segunda fase do projeto, foi essencial aprofundar os nossos conhecimentos sobre a estrutura e funcionamento dos ficheiros XML, especialmente no que diz respeito à hierarquia de grupos e à organização dos elementos gráficos. Além disso, solidificamos o que foi dado nas aulas sobre as transformações de figuras geométricas, aplicando as mesmas às figuras desenvolvidas na fase anterior, sendo estas translações, rotações e escalamentos.